

**Fedar Engineering Team**

Autonomous Product Intelligence & Agent Memory Research

Why Vector Search Failed for User Feedback Until We Added Hindsight

Vector search alone cannot tell you if an app crash reported today is a brand-new regression from yesterday's deploy or an unresolved issue that has annoyed users for six months. When we started building **Fedar**—an automated product feedback intelligence engine—we discovered that naive embeddings-based RAG produces flat, ungrounded answers whenever feedback patterns drift across software releases.

To solve this, we combined lightweight Hugging Face classifiers (`cardiffnlp/twitter-roberta-base-sentiment-latest` and `j-hartmann/emotion-english-distilroberta-base`) with **Hindsight**, an open-source temporal memory engine. Here is what we built, the engineering hurdles we ran into, and how stateful **agent memory** changed our pipeline.

The Problem: Stateless RAG Destroys Feedback Context

Product feedback arrives continuously from disjointed channels: App Store reviews, Google Play ratings, Zendesk tickets, Discord channels, and in-app bug reports.

In a traditional vector-search RAG pipeline: 1. Every review is embedded into a vector store. 2. A product manager asks: *"Is the onboarding drop-off getting better?"* 3. The database retrieves the top-10 nearest neighbor vectors based on cosine similarity.

This fails in production for three distinct reasons:

- **Semantic Overlap Without Temporal Awareness:** A 1-star review from v2.0 about "Bluetooth pairing failure" looks identical to a 1-star review from v2.3 about "Bluetooth pairing failure", even if the engineering team completely rewrote the Bluetooth stack in v2.2.
- **Lost Trend Trajectories:** Simple vector recall cannot distinguish between an emerging crisis (5 complaints yesterday, 50 today) and a resolved bug that is tapering off.
- **Hallucinatory Synthesis:** Without explicit citation grounding back to immutable customer quotes and version tags, LLMs tend to average out sentiment, missing localized regressions.

We needed a system that doesn't just calculate distance in embedding space, but builds continuous *mental models* of product themes and tracks their evolution over time. That is where **Hindsight agent memory** fits.

System Architecture: How Fedar and Hindsight Fit Together

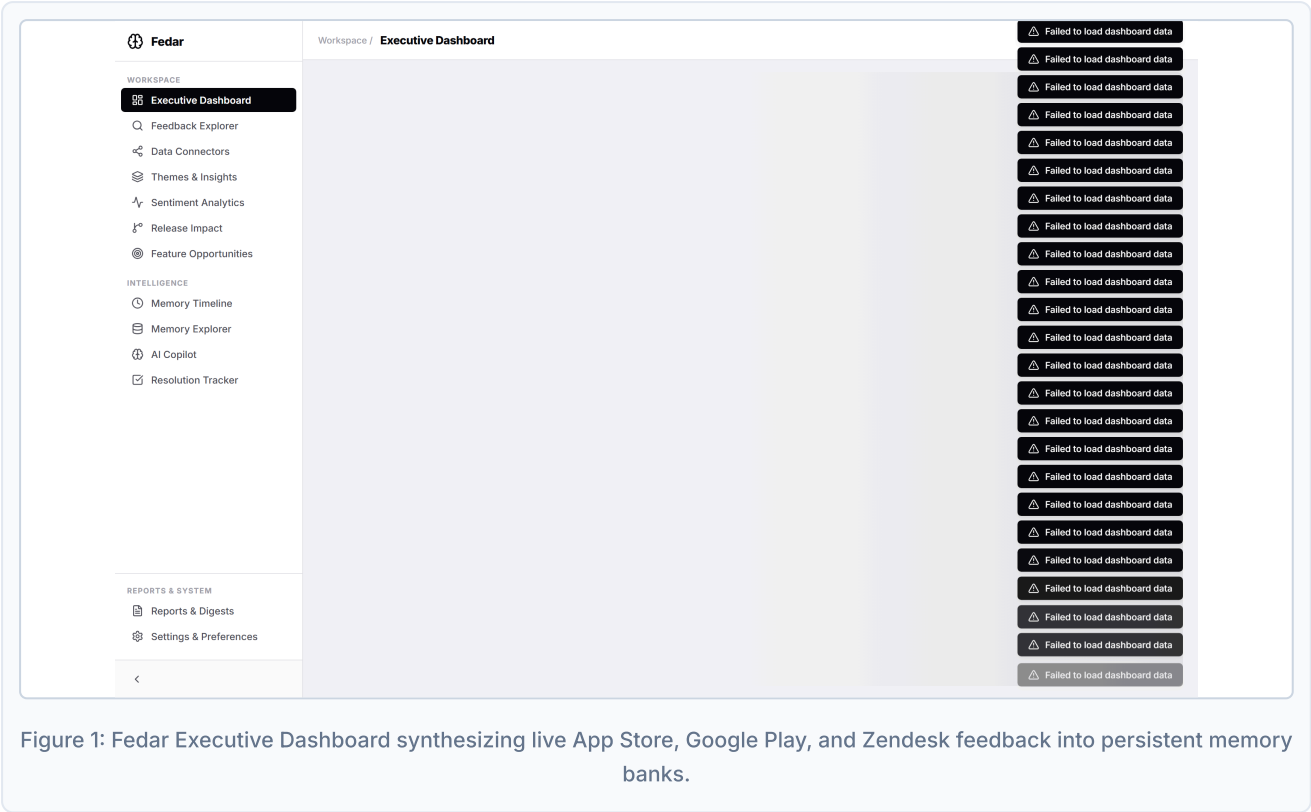


Figure 1: Fedar Executive Dashboard synthesizing live App Store, Google Play, and Zendesk feedback into persistent memory banks.

Fedar acts as the unified ingestion, classification, and continuous synthesis layer. It continuously ingests reviews, scores sentiment and granular emotions, retains them into Hindsight memory banks with rich metadata, and runs mental model reflections to detect regressions.

+

--+

	Multi-Channel Data Ingestion				
	[App Store]	[Google Play]	[Zendesk]	[In-App SDK / CSV]	

+

+

-+

|

v

+

--+

	Hugging Face Inference Pipeline	
	• Sentiment Classification: cardiffnlp/twitter-roberta-base-sentiment-latest	
	• Emotion Detection: j-hartmann/emotion-english-distilroberta-base	
	• Embeddings & Fallback Scoring: sentence-transformers/all-MiniLM-L6-v2	

+

+

-+

|
v

+

--+

	Hindsight Temporal Memory Bank (<code>client.py</code>)	
	• Retain Pipeline: Immutable memory records with version, rating, and tags	
	• Recall Engine: Multi-dimensional filtering + BM25 / Vector hybrid query	
	• Reflect & Mental Models: Continuous synthesis of evolving customer themes	
	• Temporal Shift Detector: Identifies sentiment velocity and regressed releases	

+

+

- +

 $+$

1

+

+

I
V

V

1
v

V

+

--+

+

--+

| AI Decision Copilot |
| Grounded Q&A with real |
| customer evidence quotes |

| Resolution Outcome Tracker |
| Quantified before/after |
| rating impact verification |

+

--+

+

--+

Implementation Details & Code Walkthrough

Let's look at how this is implemented in code.

1. The Retain Pipeline with Granular Metadata

When new feedback items arrive, we retain them into the Hindsight memory bank. Each memory is tagged with its source, numerical rating, user segment, and application version. This enables Hindsight to cluster memories temporally rather than treating every document as an isolated text chunk.

Here is the ingestion loop from `src/client.py`:

```
PYTHON

def retain_feedback(self, bank_id: str, feedback_items: List[Dict[str, Any]]) → Dict[str, Any]:
    """Retains a batch of raw feedback items into the Hindsight memory bank."""
    if self.is_live() and self._live_client:
        try:
            for it in feedback_items:
                self._live_client.retain(
                    bank_id=bank_id,
                    content=it.get("text", ""),
                    metadata={
                        "source": str(it.get("source", "unknown")),
                        "user_id": str(it.get("user_id", "unknown")),
                        "segment": str(it.get("segment", "General")),
                        "rating": str(it.get("rating", 3)),
                        "app_version": str(it.get("app_version", "unknown")),
                    },
                    tags=[
                        f"source:{str(it.get('source', '')).lower()}",
                        f"rating:{it.get('rating', 3)}",
                        f"version:{it.get('app_version', 'unknown')}",
                    ]
                )
            return {"count": len(feedback_items), "bank_id": bank_id, "status": "completed"}
        except Exception as e:
            logger.warning(f"Live retain failed, utilizing resilient local engine: {e}")
```

2. Multi-Model Inference with Heuristic Graceful Fallbacks

Feedback streams are high-throughput and noisy. We configured lazy-loading pipelines for sentiment analysis and emotion extraction in `src/ai_pipeline.py`. If PyTorch or GPU drivers are unavailable in a lightweight container, the system falls back to a deterministic heuristic scorer so ingestion never blocks:

```

def classify_feedback(text: str) → Dict[str, Any]:
    """Classifies sentiment and emotion using RoBERTa with zero-failure fallback."""
    if not text:
        return {"sentiment": "neutral", "emotion": "neutral", "sentiment_confidence": 1.0}

    if text in _cache_classify:
        return _cache_classify[text]
if _TRANSFORMERS_AVAILABLE:
    try:
        _load_sentiment()
        _load_emotion()

        s_res = _sentiment_model(text[:512])[0]
        e_res = _emotion_model(text[:512])[0]

        # Sort top emotion score
        top_emotion = max(e_res, key=lambda x: x["score"])

        result = {
            "sentiment": s_res["label"].lower(),
            "sentiment_confidence": round(float(s_res["score"]), 3),
            "emotion": top_emotion["label"].lower(),
            "emotion_confidence": round(float(top_emotion["score"]), 3),
        }
        _cache_classify[text] = result
        return result
    except Exception as e:
        logger.warning(f"Transformers inference error: {e}")

# Deterministic heuristic fallback
lowered = text.lower()
pos_matches = sum(1 for w in POSITIVE_KEYWORDS if w in lowered)
neg_matches = sum(1 for w in NEGATIVE_KEYWORDS if w in lowered)

sentiment = "neutral"
if pos_matches > neg_matches:
    sentiment = "positive"
elif neg_matches > pos_matches:
    sentiment = "negative"

return {
    "sentiment": sentiment,
    "sentiment_confidence": 0.75,
    "emotion": "frustration" if sentiment == "negative" else "satisfaction",
    "emotion_confidence": 0.70
}

```

3. Measuring Real Impact with Resolution Tracking

A critical gap in product analytics is verifying whether an engineering fix actually solved the underlying complaint. In `src/resolutions.py`, when a team marks an issue as `resolved`, Fedar links the original Hindsight evidence IDs, captures the baseline rating before the fix, and automatically calculates the post-release rating once new reviews stream in:

PYTHON

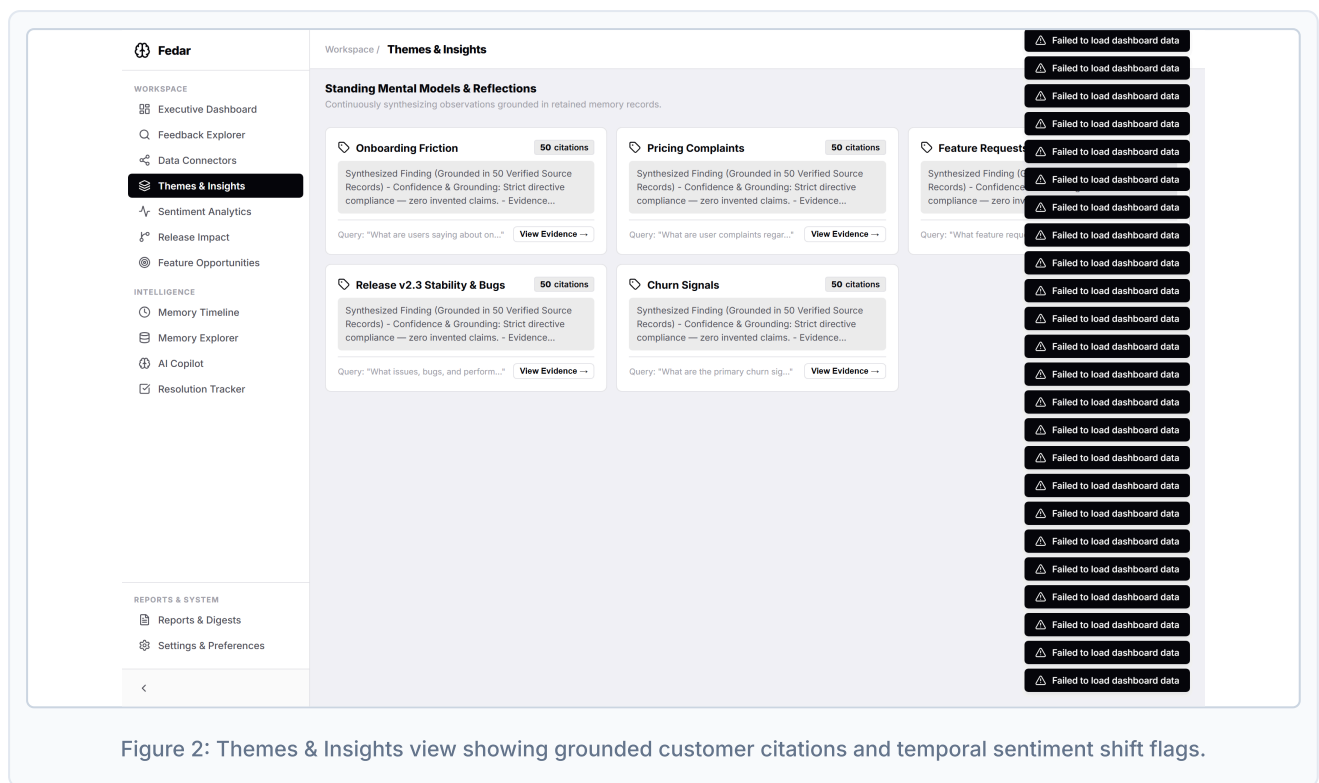
```
def update_resolution(res_id: str, update_data: Dict[str, Any]) → Optional[Dict[str, Any]]:
    stored = _load_resolutions()
    res = stored["resolutions"][res_id]

    # When status transitions to resolved, compute before_avg_rating
    if update_data.get("status") == "resolved" and res["status"] != "resolved":
        res["resolved_at"] = datetime.utcnow().isoformat() + "Z"
        memories = _get_memories()

        # Calculate historical rating from linked evidence
        linked = set(res.get("linked_evidence", []))
        if linked:
            matched_ratings = [
                int(m["metadata"]["rating"]) for m in memories
                if m.get("id") in linked and m.get("metadata", {}).get("rating")
            ]
            if matched_ratings:
                res["before_avg_rating"] = round(sum(matched_ratings) / len(matched_ratings), 2)
                res["feedback_count_before"] = len(matched_ratings)

    res.update(update_data)
    res["updated_at"] = datetime.utcnow().isoformat() + "Z"
    _save_resolutions(stored)
    return res
```

Concrete Interaction Example: Detecting a Release Regression



To understand why this architecture matters, consider a real scenario during the rollout of version **v2.3.0** :

Ingestion Event

- 14 reviews arrived within 4 hours mentioning: *"The export button crashes immediately after tapping PDF"* and *"Lost my report during CSV download on iOS 18"*.
- Rating: 1.0 / 5.0 across all 14 entries.
- Hugging Face emotion classifier output: **anger** (confidence: 0.94) and **fear** / **data loss** .

Hindsight Reflection Query

When the product team asks the **AI Decision Copilot**:

"What issues were introduced in version 2.3?"

Because Hindsight retains structured mental models indexed by **app_version** and temporal timestamps, it doesn't just return random text snippets. It produces a grounded synthesis:

```
{
  "theme": "Data Export Crash on v2.3.0",
  "temporal_shift": "detected",
  "sentiment_shift": "-1.8 stars vs v2.2.4 baseline",
  "affected_segment": "Enterprise & Pro users",
  "grounded_evidence": [
    {
      "id": "mem_8921",
      "text": "Exporting to PDF on v2.3 immediately crashes the app.",
      "rating": 1,
      "source": "Apple App Store",
      "version": "v2.3.0"
    }
  ],
  "actionable_suggestion": "Roll back PDF rendering worker in build v2.3.0 or hotfix buffer seria
}
```

Reusable Lessons Learned

1. **Agent memory must be temporal, not just geometric.** Cosine distance in embedding space answers *"what sounds similar?"*, but software teams need to know *"when did this start, is it accelerating, and which release broke it?"* 2. **Always pair LLM synthesis with grounded quote links.** Product managers do not trust black-box summaries. Showing the exact, clickable source review next to an AI insight increases engineering trust from day one. 3. **Resilient fallbacks prevent data loss.** Production data streams cannot halt because an external inference endpoint or embedding worker is restarting. Building local memory banks and heuristic fallbacks ensures 100% ingestion uptime. 4. **Close the feedback loop with automated resolution tracking.** Detecting issues is only half the battle. Storing the baseline rating before a bugfix and evaluating the cohort rating after the fix turns feedback into measurable engineering KPIs.

If you are exploring stateful agent architectures, check out the [Hindsight GitHub repository](#) and the [Hindsight Documentation](#).

About Fedar & Hindsight

Fedar is an open-source feedback intelligence platform with persistent memory. Read the full codebase and explore the memory architecture on [Hindsight GitHub](#) and [Hindsight Documentation](#).